

Online Dynamic Mode Decomposition for Time-Varying Systems*

Hao Zhang[†], Clarence W. Rowley[†], Eric A. Deem[‡], and Louis N. Cattafesta[‡]

Abstract. Dynamic mode decomposition (DMD) is a popular technique for modal decomposition, flow analysis, and reduced-order modeling. In situations where a system is time varying, one would like to update the system's description online as time evolves. This work provides an efficient method for computing DMD in real time, updating the approximation of a system's dynamics as new data becomes available. The algorithm does not require storage of past data and computes the exact DMD matrix using rank-1 updates. A weighting factor that places less weight on older data can be incorporated in a straightforward manner, making the method particularly well suited to time-varying systems. A variant of the method may also be applied to online computation of "windowed DMD," in which only the most recent data are used. The efficiency of the method is compared against several existing DMD algorithms: for problems in which the state dimension is less than about 200, the proposed algorithm is the most efficient for real-time computation, and it can be orders of magnitude more efficient than the standard DMD algorithm. The method is demonstrated on several examples, including a time-varying linear system and a more complex example using data from a wind tunnel experiment. In particular, we show that the method is effective at capturing the dynamics of surface pressure measurements in the flow over a flat plate with an unsteady separation bubble.

Key words. data-driven modeling, dynamic mode decomposition, online methods, rank-1 updating algorithm, system identification, time-varying systems

AMS subject classifications. 37M10, 93B30, 37N10, 37N35, 65P99, 37E99

DOI. 10.1137/18M1192329

1. Introduction. Modal decomposition methods are widely used in studying complex dynamical systems such as fluid flows. In particular, dynamic mode decomposition (DMD) [25, 24] has become increasingly popular in the fluids community. DMD decomposes spatio-temporal data into spatial modes (DMD modes) each of which has simple temporal behavior characterized by single frequency and growth/decay rate (DMD eigenvalues). DMD has been successfully applied to a wide range of problems, for instance, as discussed in [23, 18]. The idea of DMD is to fit a linear system to observed dynamics. However, DMD is also a promising technique for nonlinear systems, as it has been shown to be a finite-dimensional approximation to the Koopman operator, an infinite-dimensional linear operator that captures the full behavior of a nonlinear dynamical system [24, 27]. A few methods [33, 8] have been proposed to assess the accuracy (quality) of the DMD approximated Koopman eigenfunctions/eigenvalues.

*Received by the editors June 5, 2018; accepted for publication (in revised form) by T. Sauer July 4, 2019; published electronically September 12, 2019.

<https://doi.org/10.1137/18M1192329>

Funding: This work was supported by AFOSR through grant FA9550-14-1-0289 and DARPA through award HR0011-16-C-0116.

[†]Mechanical and Aerospace Engineering, Princeton University, Princeton, NJ 08544 (haozhang@princeton.edu, cwrowley@princeton.edu).

[‡]Mechanical Engineering, Florida State University, Tallahassee, FL 32310 (edeem@fsu.edu, lcattafesta@fsu.edu).

Recently, several algorithms have been proposed to compute DMD modes efficiently for very large datasets, for instance, using randomized methods [10, 9]. In situations in which the incoming data is “streaming” in nature, and one does not wish to store all of the data, a “streaming DMD” algorithm performs online updating of the DMD modes and eigenvalues [16]. Streaming DMD keeps track of a small number of orthogonal basis vectors and updates the DMD matrix projected onto the corresponding subspace. Another related method uses an incremental SVD algorithm to compute DMD modes on the fly [20]. The work proposed here may be viewed as an alternative to streaming DMD, in that we provide a method for updating the DMD matrix in real time, without the need to store all the raw data. Our method differs from streaming DMD in that we compute the exact DMD matrix, rather than a projection onto basis functions; in addition, we propose various methods for better approximation of time-varying dynamics, in particular by “forgetting” older snapshots, or giving them less weight than more recent snapshots.

It is worth emphasizing that our proposed algorithm relies on an important assumption: the number of snapshots is much larger than the state dimension. In practice, DMD is often applied to fluid problems for which the opposite is true: the state dimension is high and is much larger than the number of snapshots. In this paper, we explore an algorithm for real-time updating of the linear model (DMD matrix), with the ultimate goal of real-time modeling and control. For offline data analysis, one can have access to datasets with large numbers of states (e.g., measurements of the full flow field). However, in real-time modeling, it is often the case that we have only a small number of measurements (for instance, pressure measurements from an array of sensors). Given this limited amount of information, we are trying to find adaptive real-time models.

The paper is organized as follows. In section 2, we give an overview of DMD and describe the online DMD algorithm. In section 3, we discuss a variant called windowed DMD, in which only the most recent data are used. In section 4, we briefly describe how these methods may be used in online system identification, and in section 5 we compare the different algorithms on various examples.

2. Online dynamic mode decomposition.

2.1. The problem. We first give a brief summary of the standard DMD algorithm, as described in [27]. Suppose we have a discrete-time dynamical system given by

$$\mathbf{x}_{j+1} = \mathbf{F}(\mathbf{x}_j),$$

where $\mathbf{x}_j \in \mathbb{R}^n$ is the state vector, and $\mathbf{F} : \mathbb{R}^n \rightarrow \mathbb{R}^n$ defines the dynamics. For a given state $\mathbf{x}_j$, let $\mathbf{y}_j = \mathbf{F}(\mathbf{x}_j)$; we call $(\mathbf{x}_j, \mathbf{y}_j)$ a *snapshot pair*. For DMD, we assume we have access to a collection of snapshot pairs $(\mathbf{x}_j, \mathbf{y}_j)$ for $j = 1, \dots, k$. (It is often the case that $\mathbf{x}_{j+1} = \mathbf{y}_j$, corresponding to a sequence of points along a single trajectory, but this is not required.)

DMD seeks to find a matrix $\mathbf{A}$ such that $\mathbf{y}_j = \mathbf{A}\mathbf{x}_j$, in an approximate sense. DMD modes are then eigenvectors of the matrix $\mathbf{A}$, and DMD eigenvalues are the corresponding eigenvalues. In the present work, we are interested in obtaining a matrix $\mathbf{A}$ that varies in time, giving us a local linear model for the dynamics, but in the standard DMD approach, one seeks a single matrix $\mathbf{A}$.

Given snapshot pairs $(\mathbf{x}_j, \mathbf{y}_j)$ for $j = 1, \dots, k$, we form matrices

$$(1) \quad \mathbf{X}_k = [\mathbf{x}_1 \quad \mathbf{x}_2 \quad \cdots \quad \mathbf{x}_k], \quad \mathbf{Y}_k = [\mathbf{y}_1 \quad \mathbf{y}_2 \quad \cdots \quad \mathbf{y}_k],$$

which both have dimension $n \times k$. We wish to find an $n \times n$ matrix $\mathbf{A}_k$ such that $\mathbf{A}_k \mathbf{X}_k = \mathbf{Y}_k$ approximately holds; in particular, we are interested in the *overconstrained* problem, in which $k > n$. When the problem is *underconstrained*, the model will tend to overfit the data, and any noise present in the data will lead to poor performance of the model [4]. Note that DMD has typically been used on underconstrained problems, in which the number of states is greater than the number of snapshots (e.g., in [27]), while in this paper we consider the overconstrained case. In either case, the DMD matrix $\mathbf{A}_k$ is found by minimizing the cost function [25, 24]

$$(2) \quad J_k = \sum_{i=1}^k \|\mathbf{y}_i - \mathbf{A}_k \mathbf{x}_i\|^2 = \|\mathbf{Y}_k - \mathbf{A}_k \mathbf{X}_k\|_F^2,$$

where $\|\cdot\|$ denotes the Euclidean norm on vectors and $\|\cdot\|_F$ denotes the Frobenius norm on matrices. The unique minimum-norm solution to this least-squares problem is given by

$$(3) \quad \mathbf{A}_k = \mathbf{Y}_k \mathbf{X}_k^+,$$

where $\mathbf{X}_k^+$ denotes the Moore–Penrose pseudoinverse of $\mathbf{X}_k$.

Here, we shall assume that $\mathbf{X}_k$ has full row rank, in which case $\mathbf{X}_k \mathbf{X}_k^T$ is invertible, and

$$(4) \quad \mathbf{X}_k^+ = \mathbf{X}_k^T (\mathbf{X}_k \mathbf{X}_k^T)^{-1}.$$

This assumption is essential for the development of the online algorithm, as we shall see shortly. Under this assumption, the $\mathbf{A}_k$ given above is the unique solution that minimizes J_k . This corresponds to the case in which the number of snapshots k is large, compared with the state dimension n .

Our primary focus here is systems that may be slowly varying in time, so that the matrix $\mathbf{A}_k$ should evolve as k increases. In the following section, we will present an efficient algorithm for updating $\mathbf{A}_k$ as more data becomes available. Furthermore, if the system is time varying, it may make sense to weight more recent snapshots more heavily than less recent snapshots. In this spirit, we will consider minimizing a modified cost function

$$(5) \quad \tilde{J}_k = \sum_{i=1}^k \rho^{k-i} \|\mathbf{y}_i - \mathbf{A}_k \mathbf{x}_i\|^2$$

for some constant ρ with $0 < \rho \leq 1$. When $\rho = 1$, this cost function is the same as (2), and when $\rho < 1$, errors in past snapshots are discounted. Our algorithm will apply to this minimization problem as well, with only minor modifications and no increase in computational effort.

A sketch of the online DMD setup is shown in Figure 1. Suppose we have already computed $\mathbf{A}_k$ for a given dataset. As time progresses and a new pair of snapshots $(\mathbf{x}_{k+1}, \mathbf{y}_{k+1})$ becomes available, the matrix $\mathbf{A}_{k+1}$ may be updated according to the formula given in (3). If $\mathbf{A}_{k+1}$ is computed directly in this manner, we call this the “standard approach.”

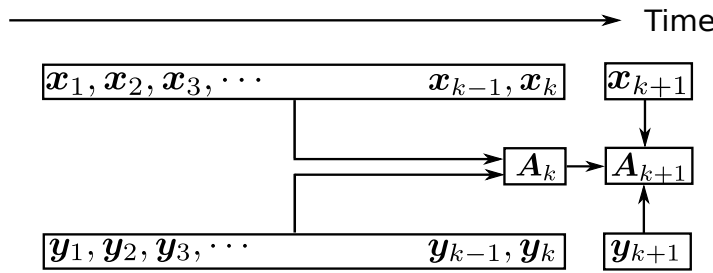


Figure 1. A cartoon of the online DMD setup. A_k is the optimal (least-squares) fit that maps $X_k = [x_1, x_2, \dots, x_k]$ to $Y_k = [y_1, y_2, \dots, y_k]$. Arrow indicates the information flow, and box denotes block of information. At time $k+1$, A_k is updated to find A_{k+1} , using the information from time k , and new available snapshot pair x_{k+1}, y_{k+1} at time $k+1$. A_{k+1} is the optimal (least-squares) fit that maps $X_{k+1} = [x_1, x_2, \dots, x_k, x_{k+1}]$ to $Y_{k+1} = [y_1, y_2, \dots, y_k, y_{k+1}]$.

There are two drawbacks to the “standard approach.” First, it requires computing the pseudoinverse of X_k whenever new snapshots are acquired, and for this reason it is computationally expensive. In addition, the method requires storing all the snapshots (i.e., storing the matrix X_k), which may be challenging or impossible as the number of snapshots k increases.

2.2. Algorithm for online DMD. To overcome the above two shortcomings, we propose a different approach to find the solution to (3) in the “online setting,” in which we want to compute A_{k+1} given a matrix A_k and a new pair of snapshots (x_{k+1}, y_{k+1}) . The algorithm we present is based on the idea that A_{k+1} should be close to A_k in some sense. For the online updating, our approach is similar to the classical recursive least-squares estimation algorithm as formulated in [17]. Here we focus on updating the DMD matrix in real time, but the recursive least-squares algorithm updates a vector in real time.

First, observe that, using (4), we may write (3) as

$$(6) \quad A_k = Y_k X_k^T (X_k X_k^T)^{-1} = Q_k P_k,$$

where Q_k and P_k are $n \times n$ matrices given by

$$(7a) \quad Q_k = Y_k X_k^T,$$

$$(7b) \quad P_k = (X_k X_k^T)^{-1}.$$

The condition that X_k has rank n ensures that $X_k X_k^T$ is invertible, and hence P_k is well defined. Note also that P_k is symmetric and strictly positive definite.

At time $k+1$, we wish to compute $A_{k+1} = Q_{k+1} P_{k+1}$. Clearly, Q_{k+1}, P_{k+1} are related to Q_k, P_k :

$$\begin{aligned} Q_{k+1} &= Y_{k+1} X_{k+1}^T = [Y_k \quad y_{k+1}] [X_k \quad x_{k+1}]^T = Y_k X_k^T + y_{k+1} x_{k+1}^T, \\ P_{k+1}^{-1} &= X_{k+1} X_{k+1}^T = [X_k \quad x_{k+1}] [X_k \quad x_{k+1}]^T = X_k X_k^T + x_{k+1} x_{k+1}^T. \end{aligned}$$

Because X_k already has rank n , and adding an additional column cannot reduce the rank of a matrix, it follows that X_{k+1} also has rank n , so P_{k+1} is well defined. The above equation shows that, given Q_k and P_k^{-1} , we may find Q_{k+1} and P_{k+1}^{-1} with simple rank-1 updates:

$$\begin{aligned} \mathbf{Q}_{k+1} &= \mathbf{Q}_k + \mathbf{y}_{k+1} \mathbf{x}_{k+1}^T, \\ \mathbf{P}_{k+1}^{-1} &= \mathbf{P}_k^{-1} + \mathbf{x}_{k+1} \mathbf{x}_{k+1}^T. \end{aligned}$$

The updated DMD matrix is then given by

$$(8) \quad \mathbf{A}_{k+1} = \mathbf{Q}_{k+1} \mathbf{P}_{k+1} = (\mathbf{Q}_k + \mathbf{y}_{k+1} \mathbf{x}_{k+1}^T) (\mathbf{P}_k^{-1} + \mathbf{x}_{k+1} \mathbf{x}_{k+1}^T)^{-1}.$$

Then the problem is reduced to how to find $\mathbf{P}_{k+1}$ from $\mathbf{P}_k$ in an efficient manner. Computing the inverse directly would require $\mathcal{O}(n^3)$ operations and would not be efficient. However, because $\mathbf{P}_{k+1}$ is the inverse of a rank-1 update of $\mathbf{P}_k^{-1}$, we may take advantage of a matrix inversion formula known as the Sherman–Morrison formula [26, 13].

Suppose $\mathbf{A}$ is an invertible square matrix, and $\mathbf{u}, \mathbf{v}$ are column vectors. Then $\mathbf{A} + \mathbf{u}\mathbf{v}^T$ is invertible if and only if $1 + \mathbf{v}^T \mathbf{A}^{-1} \mathbf{u} \neq 0$, and in this case, the inverse is given by the Sherman–Morrison formula

$$(9) \quad (\mathbf{A} + \mathbf{u}\mathbf{v}^T)^{-1} = \mathbf{A}^{-1} - \frac{\mathbf{A}^{-1} \mathbf{u} \mathbf{v}^T \mathbf{A}^{-1}}{1 + \mathbf{v}^T \mathbf{A}^{-1} \mathbf{u}}.$$

This formula is a special case of the more general matrix inversion lemma (or Woodbury formula) [31, 13].

Applying the formula to the expression for $\mathbf{P}_{k+1}$, we obtain

$$(10a) \quad \mathbf{P}_{k+1} = (\mathbf{P}_k^{-1} + \mathbf{x}_{k+1} \mathbf{x}_{k+1}^T)^{-1} = \mathbf{P}_k - \gamma_{k+1} \mathbf{P}_k \mathbf{x}_{k+1} \mathbf{x}_{k+1}^T \mathbf{P}_k,$$

where

$$(10b) \quad \gamma_{k+1} = \frac{1}{1 + \mathbf{x}_{k+1}^T \mathbf{P}_k \mathbf{x}_{k+1}}.$$

Note that, because $\mathbf{P}_k$ is positive definite, the scalar quantity $1 + \mathbf{x}_{k+1}^T \mathbf{P}_k \mathbf{x}_{k+1}$ is always nonzero, so the formula applies. Therefore, the updated DMD matrix may be written

$$(11) \quad \begin{aligned} \mathbf{A}_{k+1} &= (\mathbf{Q}_k + \mathbf{y}_{k+1} \mathbf{x}_{k+1}^T) (\mathbf{P}_k - \gamma_{k+1} \mathbf{P}_k \mathbf{x}_{k+1} \mathbf{x}_{k+1}^T \mathbf{P}_k) \\ &= \mathbf{Q}_k \mathbf{P}_k - \gamma_{k+1} \mathbf{Q}_k \mathbf{P}_k \mathbf{x}_{k+1} \mathbf{x}_{k+1}^T \mathbf{P}_k \\ &\quad + \mathbf{y}_{k+1} \mathbf{x}_{k+1}^T \mathbf{P}_k - \gamma_{k+1} \mathbf{y}_{k+1} \mathbf{x}_{k+1}^T \mathbf{P}_k \mathbf{x}_{k+1} \mathbf{x}_{k+1}^T \mathbf{P}_k. \end{aligned}$$

We can simplify the last two terms, since

$$\begin{aligned} \mathbf{y}_{k+1} \mathbf{x}_{k+1}^T \mathbf{P}_k - \gamma_{k+1} \mathbf{y}_{k+1} \mathbf{x}_{k+1}^T \mathbf{P}_k \mathbf{x}_{k+1} \mathbf{x}_{k+1}^T \mathbf{P}_k &= \gamma_{k+1} \mathbf{y}_{k+1} (\gamma_{k+1}^{-1} - \mathbf{x}_{k+1}^T \mathbf{P}_k \mathbf{x}_{k+1}) \mathbf{x}_{k+1}^T \mathbf{P}_k \\ &= \gamma_{k+1} \mathbf{y}_{k+1} \mathbf{x}_{k+1}^T \mathbf{P}_k, \end{aligned}$$

where we have used (10b). Substituting into (11), we obtain

$$\begin{aligned} \mathbf{A}_{k+1} &= \mathbf{Q}_k \mathbf{P}_k - \gamma_{k+1} \mathbf{Q}_k \mathbf{P}_k \mathbf{x}_{k+1} \mathbf{x}_{k+1}^T \mathbf{P}_k + \gamma_{k+1} \mathbf{y}_{k+1} \mathbf{x}_{k+1}^T \mathbf{P}_k \\ &= \mathbf{A}_k - \gamma_{k+1} \mathbf{A}_k \mathbf{x}_{k+1} \mathbf{x}_{k+1}^T \mathbf{P}_k + \gamma_{k+1} \mathbf{y}_{k+1} \mathbf{x}_{k+1}^T \mathbf{P}_k, \end{aligned}$$

and hence

$$(12) \quad \mathbf{A}_{k+1} = \mathbf{A}_k + \gamma_{k+1}(\mathbf{y}_{k+1} - \mathbf{A}_k \mathbf{x}_{k+1}) \mathbf{x}_{k+1}^T \mathbf{P}_k.$$

The above formula gives a rule for computing $\mathbf{A}_{k+1}$ given $\mathbf{A}_k$, $\mathbf{P}_k$ and the new snapshot pair $(\mathbf{x}_{k+1}, \mathbf{y}_{k+1})$. In order to use this formula recursively, we also need to compute $\mathbf{P}_{k+1}$ using (10), given $\mathbf{P}_k$ and $\mathbf{x}_{k+1}$.

There is an intuitive interpretation for the update formula (12). The quantity $(\mathbf{y}_{k+1} - \mathbf{A}_k \mathbf{x}_{k+1})$ can be considered as the prediction error from the current model $\mathbf{A}_k$, and the DMD matrix is updated by adding a term proportional to this error.

The updates in (10) and (12) together require only two matrix vector multiplications ($\mathbf{A}_k \mathbf{x}_{k+1}$ and $\mathbf{P}_k \mathbf{x}_{k+1}$, since $\mathbf{P}_k$ is symmetric), and two vector outer products, for a total of $4n^2$ floating-point multiplies. This is much more efficient than applying the standard DMD algorithm, which involves an SVD or pseudoinverse, and requires $O(kn^2)$ multiplies, where $k > n$. In our approach, two $n \times n$ matrices need to be stored ($\mathbf{A}_k$ and $\mathbf{P}_k$), but the large $n \times k$ snapshot matrices $(\mathbf{X}_k, \mathbf{Y}_k)$ do not need to be stored.

It is worth emphasizing that the update formulas (10) and (12) compute the DMD matrix $\mathbf{A}_{k+1} = \mathbf{Y}_{k+1} \mathbf{X}_{k+1}^+$ exactly (up to machine precision). That is, with exact arithmetic, our formulas give the same results as the standard DMD algorithm. The matrix $\mathbf{P}_k$ does involve “squaring up” the matrix $\mathbf{X}_k$, which could in principle lead to difficulties with numerical stability for ill-conditioned problems [32, 1]. However, we have not encountered problems with numerical stability in the examples we have tried (see section 5).

Initialization. The algorithm described above needs a starting point. In particular, to apply the updates (10) and (12), one needs the matrices $\mathbf{P}_k$ and $\mathbf{A}_k$ at time step k . The initialization technique is similar to the initialization of recursive least-squares estimation described in [17]. Two practical approaches are discussed below. The most straightforward way to initialize the algorithm is to first collect at least n snapshots (more precisely, enough snapshots so that $\mathbf{X}_k$ as defined in (1) has rank n), and then compute $\mathbf{P}_k$ and $\mathbf{A}_k$ using the standard DMD algorithm, from (6) and (7):

$$(13) \quad \mathbf{A}_k = \mathbf{Y}_k \mathbf{X}_k^+, \quad \mathbf{P}_k = (\mathbf{X}_k \mathbf{X}_k^T)^{-1}.$$

If for some reason this is not desirable, then an alternative approach is to initialize $\mathbf{A}_0$ to a random matrix (e.g., the zero matrix), and set $\mathbf{P}_0 = \alpha \mathbf{I}$, where α is a large positive scalar. Then in the limit as $\alpha \rightarrow \infty$, the matrices $\mathbf{P}_k, \mathbf{A}_k$ computed by the updates (10) and (12) converge to the true values given by (13).

Multiple snapshots. In our method, the DMD matrix $\mathbf{A}_k$ gets updated at every time step when a new snapshot pair becomes available. In principle, one could update the DMD matrix less frequently (for instance, every 10 time steps). The above derivation can be appropriately modified to handle this case, using the more general Woodbury formula (see (23)) [31, 13]. However, if s is the number of new snapshots to be incorporated, the computational cost of a single rank- s update is roughly the same as applying the rank-1 formula s times, so there does not appear to be a benefit to incorporating multiple snapshots at once.

Extensions. As is the case for most DMD algorithms (including streaming DMD), the online DMD algorithm described above applies more generally to extended DMD (EDMD)

[29], simply replacing the state observations $\mathbf{x}_k, \mathbf{y}_k$ by the corresponding vectors of observables. In addition, the algorithm can be used for real-time online system identification, including both linear and nonlinear system identification, as we shall discuss in section 4.

Summary. To summarize, the algorithm proceeds as follows:

1. Collect k snapshot pairs $(\mathbf{x}_j, \mathbf{y}_j)$, $j = 1, \dots, k$, where $k > n$ is large enough so that $\text{Rank } \mathbf{X}_k = n$ (where $\mathbf{X}_k$ is given by (1)).
2. Compute $\mathbf{A}_k$ and $\mathbf{P}_k$ from (13).
3. When a new snapshot pair $(\mathbf{x}_{k+1}, \mathbf{y}_{k+1})$ becomes available, update $\mathbf{A}_k$ and $\mathbf{P}_k$ according to (12) and (10).

Implementations of this algorithm in both MATLAB and Python are publicly available at [34].

2.3. Weighted online DMD. As mentioned previously, the online DMD algorithm described above is ideally suited to cases for which the system is varying in time, so that we want to revise our estimate of the DMD matrix $\mathbf{A}_k$ in real time. In such a situation, we might wish to place more weight on recent snapshots, and gradually “forget” the older snapshots, by minimizing a cost function of the form (5) instead of the original cost function (2). This weighting scheme is analogous to that used in real-time least-squares approximation [17]. This idea may also be used with streaming DMD and in fact has been considered before (the conference presentation [15] implemented such a “forgetting factor” with streaming DMD, although it did not appear in the associated paper). It turns out that the online DMD algorithm can be adapted to minimize the cost function (5) with only minor modifications to the algorithm.

We now consider the cost function

$$\tilde{J}_k = \sum_{i=1}^k \rho^{k-i} \|\mathbf{y}_i - \mathbf{A}_k \mathbf{x}_i\|^2, \quad 0 < \rho \leq 1,$$

where ρ is the weighting factor. For instance, if we wish our snapshots to have a “half-life” of m samples, then we could choose $\rho = 2^{-1/m}$. In practice, ρ should be chosen according to how fast the dynamics are changing. There is a trade-off between faster tracking and noise filtering: a smaller ρ will result in faster tracking, while inevitably making the identified model more sensitive to noise in the data (since we are forgetting old samples). For convenience, let us take $\rho = \sigma^2$, where $0 < \sigma \leq 1$, and write the cost function as

$$\tilde{J}_k = \sum_{i=1}^k \left\| \sigma^{k-i} \mathbf{y}_i - \mathbf{A}_k \sigma^{k-i} \mathbf{x}_i \right\|^2.$$

If we define matrices based on scaled versions of the snapshots as

$$\begin{aligned} \tilde{\mathbf{X}}_k &= [\sigma^{k-1} \mathbf{x}_1 \quad \sigma^{k-2} \mathbf{x}_2 \quad \cdots \quad \mathbf{x}_k], \\ \tilde{\mathbf{Y}}_k &= [\sigma^{k-1} \mathbf{y}_1 \quad \sigma^{k-2} \mathbf{y}_2 \quad \cdots \quad \mathbf{y}_k], \end{aligned}$$

then the cost function can be written as

$$\tilde{J}_k = \left\| \tilde{\mathbf{Y}}_k - \mathbf{A}_k \tilde{\mathbf{X}}_k \right\|_F^2.$$

The unique least-squares solution that minimizes this cost function (assuming $\tilde{\mathbf{X}}_k$ has full row rank) is given by

$$\mathbf{A}_k = \tilde{\mathbf{Y}}_k \tilde{\mathbf{X}}_k^+ = \tilde{\mathbf{Y}}_k \tilde{\mathbf{X}}_k^T \left(\tilde{\mathbf{X}}_k \tilde{\mathbf{X}}_k^T \right)^{-1} = \tilde{\mathbf{Q}}_k \tilde{\mathbf{P}}_k,$$

where we define

$$\begin{aligned} \tilde{\mathbf{Q}}_k &= \tilde{\mathbf{Y}}_k \tilde{\mathbf{X}}_k^T, \\ \tilde{\mathbf{P}}_k &= \left(\tilde{\mathbf{X}}_k \tilde{\mathbf{X}}_k^T \right)^{-1}. \end{aligned}$$

At step $k+1$, we wish to compute $\mathbf{A}_{k+1} = \tilde{\mathbf{Q}}_{k+1} \tilde{\mathbf{P}}_{k+1}$. We write down $\tilde{\mathbf{X}}_{k+1}, \tilde{\mathbf{Y}}_{k+1}$ explicitly as

$$\begin{aligned} \tilde{\mathbf{X}}_{k+1} &= [\sigma^k \mathbf{x}_1 \quad \sigma^{k-1} \mathbf{x}_2 \quad \cdots \quad \sigma \mathbf{x}_k \quad \mathbf{x}_{k+1}] = [\sigma \tilde{\mathbf{X}}_k \quad \mathbf{x}_{k+1}], \\ \tilde{\mathbf{Y}}_{k+1} &= [\sigma^k \mathbf{y}_1 \quad \sigma^{k-1} \mathbf{y}_2 \quad \cdots \quad \sigma \mathbf{y}_k \quad \mathbf{y}_{k+1}] = [\sigma \tilde{\mathbf{Y}}_k \quad \mathbf{y}_{k+1}]. \end{aligned}$$

Therefore, $\tilde{\mathbf{Q}}_{k+1}$ can be written

$$\begin{aligned} \tilde{\mathbf{Q}}_{k+1} &= \tilde{\mathbf{Y}}_{k+1} \tilde{\mathbf{X}}_{k+1}^T = [\sigma \tilde{\mathbf{Y}}_k \quad \mathbf{y}_{k+1}] [\sigma \tilde{\mathbf{X}}_k \quad \mathbf{x}_{k+1}]^T \\ &= \sigma^2 \tilde{\mathbf{Y}}_k \tilde{\mathbf{X}}_k^T + \mathbf{y}_{k+1} \mathbf{x}_{k+1}^T \\ &= \rho \tilde{\mathbf{Q}}_k + \mathbf{y}_{k+1} \mathbf{x}_{k+1}^T, \end{aligned}$$

and similarly

$$(14) \quad \tilde{\mathbf{P}}_{k+1}^{-1} = \rho \tilde{\mathbf{P}}_k^{-1} + \mathbf{x}_{k+1} \mathbf{x}_{k+1}^T.$$

The updated DMD matrix is then given by

$$\mathbf{A}_{k+1} = \tilde{\mathbf{Q}}_{k+1} \tilde{\mathbf{P}}_{k+1} = \left(\rho \tilde{\mathbf{Q}}_k + \mathbf{y}_{k+1} \mathbf{x}_{k+1}^T \right) \left(\rho \tilde{\mathbf{P}}_k^{-1} + \mathbf{x}_{k+1} \mathbf{x}_{k+1}^T \right)^{-1}.$$

As before, we can apply the Sherman–Morrison formula (9) to (14) and obtain

$$\tilde{\mathbf{P}}_{k+1} = \frac{\tilde{\mathbf{P}}_k}{\rho} - \gamma_{k+1} \frac{\tilde{\mathbf{P}}_k}{\rho} \mathbf{x}_{k+1} \mathbf{x}_{k+1}^T \frac{\tilde{\mathbf{P}}_k}{\rho},$$

where

$$\gamma_{k+1} = \frac{1}{1 + \mathbf{x}_{k+1}^T (\tilde{\mathbf{P}}_k / \rho) \mathbf{x}_{k+1}}.$$

Let us rescale $\tilde{\mathbf{P}}_k$, and define

$$\hat{\mathbf{P}}_k = \frac{\tilde{\mathbf{P}}_k}{\rho} = \frac{1}{\rho} \left(\tilde{\mathbf{X}}_k \tilde{\mathbf{X}}_k^T \right)^{-1}.$$

Then after some manipulation, the formula for $\mathbf{A}_{k+1}$ becomes

$$(15) \quad \mathbf{A}_{k+1} = \mathbf{A}_k + \gamma_{k+1} (\mathbf{y}_{k+1} - \mathbf{A}_k \mathbf{x}_{k+1}) \mathbf{x}_{k+1}^T \hat{\mathbf{P}}_k,$$

where

$$(16a) \quad \hat{\mathbf{P}}_{k+1} = \frac{1}{\rho} \left(\hat{\mathbf{P}}_k - \gamma_{k+1} \hat{\mathbf{P}}_k \mathbf{x}_{k+1} \mathbf{x}_{k+1}^T \hat{\mathbf{P}}_k \right),$$

$$(16b) \quad \gamma_{k+1} = \frac{1}{1 + \mathbf{x}_{k+1}^T \hat{\mathbf{P}}_k \mathbf{x}_{k+1}}.$$

Observe that the update (15) for $\mathbf{A}_{k+1}$ is identical to the update (12) from the previous section, with $\mathbf{P}_k$ replaced by $\hat{\mathbf{P}}_k$, and the update rule (16) for $\hat{\mathbf{P}}_{k+1}$ differs from (10) only by a factor of ρ . When $\rho = 1$, of course, the above formulas are identical to those given in subsection 2.2.

3. Windowed dynamic mode decomposition. In subsection 2.3, we presented a method for gradually “forgetting” older snapshots, by giving them less weight in a cost function. In this section, we discuss an alternative method, which uses a hard cutoff: in particular, we consider a “window” containing only the most recent snapshots, for instance, as used in [12, 19].

3.1. The problem. If the dynamics are slowly varying with time, we may wish to use only the most recent snapshots to identify the dynamics. Here, we consider the case where we use only a fixed “window” containing the most recent snapshots. Here, we present an “online” algorithm to compute windowed DMD efficiently, again using low-rank updates, as in the previous section. We refer to the resulting method as “windowed DMD.”

At time t_k , suppose we have access to past snapshot pairs $\{(\mathbf{x}_j, \mathbf{y}_j)\}_{j=k-w+1}^k$ in a finite time window of size w . We would like to fit a linear model $\mathbf{A}_k$, such that $\mathbf{y}_j = \mathbf{A}_k \mathbf{x}_j$ (at least approximately) for all j in this window. Let

$$(17) \quad \mathbf{X}_k = [\mathbf{x}_{k-w+1} \quad \mathbf{x}_{k-w+2} \quad \cdots \quad \mathbf{x}_k], \quad \mathbf{Y}_k = [\mathbf{y}_{k-w+1} \quad \mathbf{y}_{k-w+2} \quad \cdots \quad \mathbf{y}_k],$$

both $n \times w$ matrices. Then we seek an $n \times n$ matrix $\mathbf{A}_k$ such that $\mathbf{A}_k \mathbf{X}_k = \mathbf{Y}_k$ approximately holds. More precisely (as explained in subsection 2.1), the DMD matrix $\mathbf{A}_k$ is found by minimizing

$$(18) \quad J_k = \|\mathbf{Y}_k - \mathbf{A}_k \mathbf{X}_k\|_F^2.$$

As before, we assume that the rank of $\mathbf{X}_k$ is $n \leq w$, so that there is a unique solution to this least-squares problem, given by

$$(19) \quad \mathbf{A}_k = \mathbf{Y}_k \mathbf{X}_k^+,$$

where $\mathbf{X}_k^+ = \mathbf{X}_k^T (\mathbf{X}_k \mathbf{X}_k^T)^{-1}$ is the Moore–Penrose pseudoinverse of $\mathbf{X}_k$. (Note, in particular, that we require that the window size w be at least as large as the state dimension n , so that $\mathbf{X}_k \mathbf{X}_k^T$ is invertible.)

A sketch of the windowed DMD setup is shown in Figure 2. As time progresses, we can update $\mathbf{A}_k$ according to the formula given in (19). However, evaluating (19) involves computing a new pseudoinverse and a matrix multiplication, which are costly operations. We may compute this update more efficiently using an approach similar to that in the previous section, as we describe below.

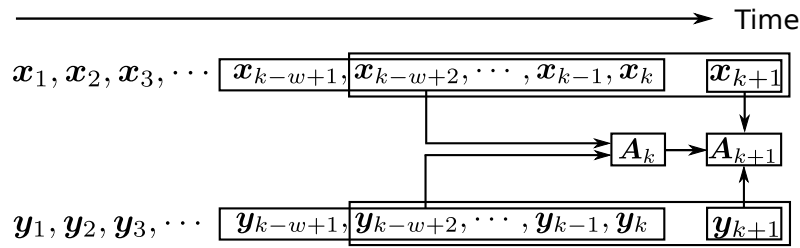


Figure 2. A cartoon of the windowed DMD setup. At time k , $\mathbf{A}_k$ depends only on the w most recent snapshots. At time $k+1$, one new snapshot is added, and the oldest snapshot is dropped.

3.2. Algorithm for windowed DMD. As in the approach presented in subsection 2.2, observe that (19) can be written as

$$(20) \quad \mathbf{A}_k = \mathbf{Y}_k \mathbf{X}_k^+ = \mathbf{Y}_k \mathbf{X}_k^T (\mathbf{X}_k \mathbf{X}_k^T)^{-1} = \mathbf{Q}_k \mathbf{P}_k,$$

where

$$(21) \quad \begin{aligned} \mathbf{Q}_k &= \mathbf{Y}_k \mathbf{X}_k^T = \sum_{i=k-w+1}^k \mathbf{y}_i \mathbf{x}_i^T, \\ \mathbf{P}_k &= (\mathbf{X}_k \mathbf{X}_k^T)^{-1} = \left(\sum_{i=k-w+1}^k \mathbf{x}_i \mathbf{x}_i^T \right)^{-1}, \end{aligned}$$

where $\mathbf{Q}_k$ and $\mathbf{P}_k$ are $n \times n$ matrices. The condition that $\mathbf{X}_k$ has rank n ensures that $\mathbf{X}_k \mathbf{X}_k^T$ is invertible, so $\mathbf{P}_k$ is well defined.

At step $k+1$, we need to compute $\mathbf{A}_{k+1} = \mathbf{Q}_{k+1} \mathbf{P}_{k+1}$. Clearly, $\mathbf{Q}_{k+1}, \mathbf{P}_{k+1}$ are related to $\mathbf{Q}_k, \mathbf{P}_k$. To show this, we write them down explicitly as

$$\begin{aligned} \mathbf{Q}_{k+1} &= \mathbf{Y}_{k+1} \mathbf{X}_{k+1}^T = \sum_{i=k-w+2}^{k+1} \mathbf{y}_i \mathbf{x}_i^T = \mathbf{Q}_k - \mathbf{y}_{k-w+1} \mathbf{x}_{k-w+1}^T + \mathbf{y}_{k+1} \mathbf{x}_{k+1}^T, \\ \mathbf{P}_{k+1}^{-1} &= \mathbf{X}_{k+1} \mathbf{X}_{k+1}^T = \sum_{i=k-w+2}^{k+1} \mathbf{x}_i \mathbf{x}_i^T = \mathbf{P}_k^{-1} - \mathbf{x}_{k-w+1} \mathbf{x}_{k-w+1}^T + \mathbf{x}_{k+1} \mathbf{x}_{k+1}^T. \end{aligned}$$

There is an intuitive interpretation to this relationship: $\mathbf{Q}_k, \mathbf{P}_k$ forgets the oldest snapshot and incorporates the newest snapshot and gets updated into $\mathbf{Q}_{k+1}, \mathbf{P}_{k+1}$. As in subsection 2.2, where we used the Sherman–Morrison formula (9) to update $\mathbf{P}_k$, we may use a similar approach to update $\mathbf{P}_k$ in this case.

Letting

$$\mathbf{U} = [\mathbf{x}_{k-w+1} \quad \mathbf{x}_{k+1}], \quad \mathbf{V} = [\mathbf{y}_{k-w+1} \quad \mathbf{y}_{k+1}], \quad \mathbf{C} = \begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix},$$

we may write $\mathbf{Q}_{k+1}, \mathbf{P}_{k+1}$ as

$$\begin{aligned} \mathbf{Q}_{k+1} &= \mathbf{Q}_k + \mathbf{V} \mathbf{C} \mathbf{U}^T, \\ \mathbf{P}_{k+1}^{-1} &= \mathbf{P}_k^{-1} + \mathbf{U} \mathbf{C} \mathbf{U}^T, \end{aligned}$$

therefore

$$(22) \quad \mathbf{A}_{k+1} = \mathbf{Q}_{k+1} \mathbf{P}_{k+1} = (\mathbf{Q}_k + \mathbf{V} \mathbf{C} \mathbf{U}^T) (\mathbf{P}_k^{-1} + \mathbf{U} \mathbf{C} \mathbf{U}^T)^{-1}.$$

Now, the matrix inversion lemma (or Woodbury formula) [31, 13] states that

$$(23) \quad (\mathbf{A} + \mathbf{U} \mathbf{C} \mathbf{V})^{-1} = \mathbf{A}^{-1} - \mathbf{A}^{-1} \mathbf{U} (\mathbf{C}^{-1} + \mathbf{V} \mathbf{A}^{-1} \mathbf{U})^{-1} \mathbf{V} \mathbf{A}$$

whenever $\mathbf{A}$, $\mathbf{C}$, and $\mathbf{A} + \mathbf{U} \mathbf{C} \mathbf{V}$ are invertible. Applying this formula to our expression for $\mathbf{P}_{k+1}$, we have

$$(24a) \quad \mathbf{P}_{k+1} = \mathbf{P}_k - \mathbf{P}_k \mathbf{U} \mathbf{\Gamma}_{k+1} \mathbf{U}^T \mathbf{P}_k,$$

where

$$(24b) \quad \mathbf{\Gamma}_{k+1} = (\mathbf{C}^{-1} + \mathbf{U}^T \mathbf{P}_k \mathbf{U})^{-1}.$$

Substituting back into (22), we obtain

$$(25) \quad \begin{aligned} \mathbf{A}_{k+1} &= (\mathbf{Q}_k + \mathbf{V} \mathbf{C} \mathbf{U}^T) (\mathbf{P}_k - \mathbf{P}_k \mathbf{U} \mathbf{\Gamma}_{k+1} \mathbf{U}^T \mathbf{P}_k) \\ &= \mathbf{Q}_k \mathbf{P}_k - \mathbf{Q}_k \mathbf{P}_k \mathbf{U} \mathbf{\Gamma}_{k+1} \mathbf{U}^T \mathbf{P}_k \\ &\quad + \mathbf{V} \mathbf{C} \mathbf{U}^T \mathbf{P}_k - \mathbf{V} \mathbf{C} \mathbf{U}^T \mathbf{P}_k \mathbf{U} \mathbf{\Gamma}_{k+1} \mathbf{U}^T \mathbf{P}_k. \end{aligned}$$

The last two terms simplify, since

$$\begin{aligned} \mathbf{V} \mathbf{C} \mathbf{U}^T \mathbf{P}_k - \mathbf{V} \mathbf{C} \mathbf{U}^T \mathbf{P}_k \mathbf{U} \mathbf{\Gamma}_{k+1} \mathbf{U}^T \mathbf{P}_k &= \mathbf{V} \mathbf{C} (\mathbf{\Gamma}_{k+1}^{-1} - \mathbf{U}^T \mathbf{P}_k \mathbf{U}) \mathbf{\Gamma}_{k+1} \mathbf{U}^T \mathbf{P}_k \\ &= \mathbf{V} \mathbf{C} \mathbf{C}^{-1} \mathbf{\Gamma}_{k+1} \mathbf{U}^T \mathbf{P}_k = \mathbf{V} \mathbf{\Gamma}_{k+1} \mathbf{U}^T \mathbf{P}_k, \end{aligned}$$

where we have used (24b). Substituting into (25), we obtain

$$\begin{aligned} \mathbf{A}_{k+1} &= \mathbf{Q}_k \mathbf{P}_k - \mathbf{Q}_k \mathbf{P}_k \mathbf{U} \mathbf{\Gamma}_{k+1} \mathbf{U}^T \mathbf{P}_k + \mathbf{V} \mathbf{\Gamma}_{k+1} \mathbf{U}^T \mathbf{P}_k \\ &= \mathbf{A}_k - \mathbf{A}_k \mathbf{U} \mathbf{\Gamma}_{k+1} \mathbf{U}^T \mathbf{P}_k + \mathbf{V} \mathbf{\Gamma}_{k+1} \mathbf{U}^T \mathbf{P}_k, \end{aligned}$$

and hence

$$(26) \quad \mathbf{A}_{k+1} = \mathbf{A}_k + (\mathbf{V} - \mathbf{A}_k \mathbf{U}) \mathbf{\Gamma}_{k+1} \mathbf{U}^T \mathbf{P}_k.$$

Notice the similarity between this expression with the updating formula (12) for online DMD. $\mathbf{\Gamma}_{k+1}$ is the matrix version of γ_{k+1} in (10b). The matrix $(\mathbf{V} - \mathbf{A}_k \mathbf{U})$ can also be considered as the prediction error based on current model $\mathbf{A}_k$, and the correction to DMD matrix is proportional to this error term.

The updates in (26), (24) require two products of $n \times n$ and $n \times 2$ matrices (to compute $\mathbf{A}_k \mathbf{U}$ and $\mathbf{P}_k \mathbf{U}$, since $\mathbf{P}_k$ is symmetric), and two products of $n \times 2$ and $2 \times n$ matrices, for a total of $8n^2$ multiplies. This windowed DMD approach is much more efficient than the standard DMD approach, solving (20) directly ($\mathcal{O}(wn^2)$ multiplies, with $w \geq n$). Windowed DMD can be initialized in the same manner as online DMD, discussed in subsection 2.2.

In order to implement windowed DMD, we need to store two $n \times n$ matrices $(\mathbf{A}_k, \mathbf{P}_k)$, as well as the w most recent snapshots. Thus, the storage required is more than in online DMD, or the weighted online DMD approach discussed in subsection 2.3, which also provides a mechanism for “forgetting” older snapshots. It is worth pointing out that the update formulas (24), (26) give the exact solution $\mathbf{A}_{k+1} = \mathbf{Y}_{k+1} \mathbf{X}_{k+1}^+$ from (22), without approximation.

Larger window stride size. We can in principle move more than one step for windowed DMD, i.e., forgetting multiple snapshots and remembering multiple snapshots. If we would like to move the sliding window for s steps ($s < n/2$), then after similar derivations we can show that the computational cost is $8sn^2$ multiplies, which is the same as applying the rank-2 formulas s times. Therefore, there is no obvious advantage to incorporating multiple snapshots at one time.

Extensions. Similar to online DMD, we can also incorporate an exponential weighting factor into windowed DMD. In particular, consider the cost function as

$$\tilde{J}_k = \sum_{i=k-w+1}^k \rho^{k-i} \|\mathbf{y}_i - \mathbf{A}_k \mathbf{x}_i\|^2, \quad 0 < \rho \leq 1,$$

where ρ is the weighting factor. Then, proceeding as in [subsection 2.3](#), we obtain the update formulas

$$(27) \quad \mathbf{A}_{k+1} = \mathbf{A}_k + (\mathbf{V} - \mathbf{A}_k \mathbf{U}) \tilde{\Gamma}_{k+1} \mathbf{U}^T \hat{\mathbf{P}}_k,$$

$$(28a) \quad \hat{\mathbf{P}}_{k+1} = \frac{1}{\rho} (\hat{\mathbf{P}}_k - \hat{\mathbf{P}}_k \mathbf{U} \tilde{\Gamma}_{k+1} \mathbf{U}^T \hat{\mathbf{P}}_k),$$

where

$$(28b) \quad \tilde{\Gamma}_{k+1} = (\tilde{\mathbf{C}}^{-1} + \mathbf{U}^T \hat{\mathbf{P}}_k \mathbf{U})^{-1}, \quad \tilde{\mathbf{C}} = \begin{bmatrix} -\rho^w & 0 \\ 0 & 1 \end{bmatrix}.$$

As with the online DMD algorithm, the above windowed DMD algorithm applies generally to EDMD [29] as well, if $\mathbf{x}_k, \mathbf{y}_k$ are simply replaced by the observable vector of the states. In addition, the algorithm can be used for real-time online system identification, including both linear and nonlinear system identification, as discussed in [section 4](#).

Summary. To summarize, the algorithm proceeds as follows:

1. Collect w snapshot pairs $(\mathbf{x}_j, \mathbf{y}_j)$, $j = 1, \dots, w$, where $w \geq n$ is large enough so that $\text{Rank } \mathbf{X}_k = n$ (where $\mathbf{X}_k$ is given by (17)).
2. Compute $\mathbf{A}_k$ and $\mathbf{P}_k$ from (13), where $\mathbf{X}_k, \mathbf{Y}_k$ is given by (17).
3. When a new snapshot pair $(\mathbf{x}_{k+1}, \mathbf{y}_{k+1})$ becomes available, update $\mathbf{A}_k$ and $\mathbf{P}_k$ according to (26) and (24).

Implementations of this algorithm in both MATLAB and Python are publicly available at [34].

4. Online system identification. As previously mentioned, the online and windowed DMD algorithms discussed above can be generalized to online system identification with control in a straightforward manner. For a review of system identification methods, see [2].

4.1. Online linear system identification. DMD can be used for system identification, as shown in [22]. Suppose we are interested in identifying a (discrete-time) linear system given by

$$(29) \quad \mathbf{x}_{k+1} = \mathbf{A} \mathbf{x}_k + \mathbf{B} \mathbf{u}_k,$$

where $\mathbf{x}_k \in \mathbb{R}^n, \mathbf{u}_k \in \mathbb{R}^p$ are the states and control input, respectively, $\mathbf{A} \in \mathbb{R}^{n \times n}, \mathbf{B} \in \mathbb{R}^{n \times p}$.

At time k , assume that we have access to $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_{k+1}$ and $\mathbf{u}_1, \mathbf{u}_2, \dots, \mathbf{u}_k$. Letting

$$\tilde{\mathbf{Y}}_k = [\mathbf{x}_2 \quad \mathbf{x}_3 \quad \cdots \quad \mathbf{x}_{k+1}], \quad \tilde{\mathbf{X}}_k = \begin{bmatrix} \mathbf{x}_1 & \mathbf{x}_2 & \cdots & \mathbf{x}_k \\ \mathbf{u}_1 & \mathbf{u}_2 & \cdots & \mathbf{u}_k \end{bmatrix}, \quad \tilde{\mathbf{A}} = [\mathbf{A} \quad \mathbf{B}],$$

we may write (29) in the form

$$(30) \quad \tilde{\mathbf{Y}}_k = \tilde{\mathbf{A}} \tilde{\mathbf{X}}_k.$$

The matrices $\mathbf{A}, \mathbf{B}$ may then be found by minimizing the cost function

$$(31) \quad J_k = \|\tilde{\mathbf{Y}}_k - \tilde{\mathbf{A}}_k \tilde{\mathbf{X}}_k\|_F^2.$$

As before, the solution is given by

$$(32) \quad \tilde{\mathbf{A}}_k = \tilde{\mathbf{Y}}_k \tilde{\mathbf{X}}_k^+.$$

At time $k+1$, we add a new column to $\tilde{\mathbf{X}}_k$ and $\tilde{\mathbf{Y}}_k$, and we would like to update $\tilde{\mathbf{A}}_{k+1}$ using our previous knowledge of $\tilde{\mathbf{A}}_k$. Using the same approach as in section 2, it is straightforward to extend the online DMD and windowed DMD algorithms to this case. In particular, the square matrix $\mathbf{A}_k$ from section 2 is replaced by the rectangular matrix $\tilde{\mathbf{A}}_k$ defined above, and the vector $\mathbf{x}_k$ in the formulas in section 2 is replaced by the column vector

$$\begin{bmatrix} \mathbf{x}_k \\ \mathbf{u}_k \end{bmatrix}.$$

4.2. Online nonlinear system identification. The efficient online/windowed DMD algorithms apply to nonlinear system identification as well. In general, nonlinear system identification is a challenging problem; see [21] for an overview. Some interesting methods are to use linear-parameter-varying models [28, 14], or to consider a large dictionary of potential nonlinear functions, and exploit sparsity to select a small subset [5].

Suppose we are interested in identifying a nonlinear system

$$\mathbf{x}_{k+1} = \mathbf{f}(\mathbf{x}_k, \mathbf{u}_k)$$

directly from data, where $\mathbf{x}_k \in \mathbb{R}^n$, $\mathbf{u}_k \in \mathbb{R}^p$ are the state vector and control input, respectively. The specific form of nonlinearity is unknown, but in order to proceed, we have to make some assumptions about the nonlinear form. Assume that we have q (nonlinear) observables $z_i(\mathbf{x}, \mathbf{u})$, $i = 1, 2, \dots, q$, such that the underlying dynamics can be approximately described by

$$(33) \quad \mathbf{x}_{k+1} = \mathbf{A} \mathbf{z}_k,$$

where $\mathbf{A} \in \mathbb{R}^{n \times q}$, and

$$\mathbf{z}_k = [z_1(\mathbf{x}_k, \mathbf{u}_k) \quad z_2(\mathbf{x}_k, \mathbf{u}_k) \quad \cdots \quad z_q(\mathbf{x}_k, \mathbf{u}_k)]^T.$$

To illustrate how this representation works, we take $\mathbf{x} \in \mathbb{R}$, $\mathbf{u} \in \mathbb{R}$, for example, and assume the nonlinear dynamics is given by

$$x_{k+1} = a_1 x_k + a_2 x_k^2 + a_3 u_k + a_4 u_k^2 + a_5 x_k u_k.$$

Then by setting $z_1(x, u) = x$, $z_2(x, u) = x^2$, $z_3(x, u) = u$, $z_4(x, u) = u^2$, $z_5(x, u) = xu$, we can write the dynamics in the form (33), with

$$\mathbf{A} = \begin{bmatrix} a_1 & a_2 & a_3 & a_4 & a_5 \end{bmatrix}.$$

Note that in the above, the state x_k still evolves nonlinearly (i.e., x_{k+1} depends in a nonlinear way on x_k and u_k), but we are able to identify the coefficients a_k using linear regression (i.e., finding the matrix $\mathbf{A}$ in (33)).

This approach is related to Carleman linearization [3, 11], although in Carleman linearization, the goal is to find a true linear representation of the dynamics in a higher-dimensional state space, and for most nonlinear systems of practical interest, it is not possible to obtain a finite-dimensional linear representation. This is also related to EDMD [29] and kernel DMD [30], where Koopman eigenfunctions are used to determine coordinates in which the nonlinear system becomes linear. Our approach is to define a collection of observables in order to write the original nonlinear system in a linear fashion. It is generally difficult to determine what observables to use, and sparsity-promoting system identification [5] is one possible method. If one knows something about the form of the nonlinearity (e.g., the nonlinear terms are quadratic), then this can inform the choice of observables (e.g., including all quadratic couplings of the states).

In summary, by assuming a particular form of the nonlinearity, we can find the coefficients of a nonlinear system using the same techniques as used in linear system identification, writing the nonlinear system in the form (33).

5. Application and results. In this section, we illustrate the methods on a number of examples, first showing results for simple benchmark problems, and then using data from a wind tunnel experiment.

5.1. Benchmarks. We now present a study of the computational time of various DMD algorithms. Two benchmark tasks are considered here. In the first task, we wish to know the DMD matrix only at the final time step, at which point we have access to all of the data. In the second task, we wish to compute the DMD matrix at each time, whenever a new snapshot is acquired. The first task thus represents the standard approach to computing the DMD matrix, while the second task applies to situations where the system is time varying, and we wish to update the DMD matrix in real time.

Asymptotic cost. First, we examine how the various algorithms scale with the state dimension n and the number of snapshots m , for the two tasks described above. In particular, we are concerned with the overconstrained case in which $n < m$. For the standard algorithm, in which the DMD matrix is computed directly using (3), one must compute an $n \times m$ pseudoinverse and an $n \times m$, $m \times n$ matrix multiplication. For the first task, the computational cost (measured by the number of multiplies) is thus

$$T_{\text{standard}} = \mathcal{O}(nm \min(m, n) + mn^2) = \mathcal{O}(mn^2).$$

For the second task, in which we compute the DMD matrix at each time, we refer to the standard algorithm as “batch DMD,” since the snapshots are processed all in one batch. The

method is initialized and applied after m_0 snapshots are gathered (and in the examples below, we will take $m_0 = n$), so the computational cost is

$$T_{\text{batch}} = \mathcal{O} \left(\sum_{k=m_0}^m (nk \min(k, n) + kn^2) \right) = \mathcal{O}(m^2 n^2).$$

Next, we consider windowed DMD, for a window containing w snapshots (with $n < w < m$). In this case, we refer to the standard algorithm, in which the DMD matrix is computed directly using (19), as “mini-batch DMD.” The computational cost is given by

$$T_{\text{mini-batch}} = \mathcal{O} \left(\sum_{k=w}^m (nw \min(n, w) + wn^2) \right) = \mathcal{O}(mwn^2).$$

For streaming DMD [16] for a fixed rank r , the cost of one iteration is $\mathcal{O}(r^2 n)$, and for full-rank streaming DMD, the cost of one iteration is $\mathcal{O}(n^2)$. Thus, for either task, the overall cost after m snapshots is

$$T_{\text{streaming}}^{r=n} = \mathcal{O} \left(\sum_{k=1}^m n^2 \right) = \mathcal{O}(mn^2)$$

and

$$T_{\text{streaming}}^{r<n} = \mathcal{O} \left(\sum_{k=1}^m r^2 n \right) = \mathcal{O}(mr^2 n).$$

(If, in streaming DMD, the compression step (step 3 in the algorithm in [16]) is performed only every r steps, then the cost is reduced to $\mathcal{O}(mrn)$.) Finally, for both online and windowed DMD algorithms, discussed in subsections 2.2 and 3.2, the cost per time step is $\mathcal{O}(n^2)$. The algorithms are applied after w snapshots are gathered, so the overall cost of either algorithm is

$$T_{\text{online}} = T_{\text{window}} = \mathcal{O} \left(\sum_{k=w+1}^m n^2 \right) = \mathcal{O}(mn^2).$$

Results. We now compare the performance of the different algorithms on actual examples for the two tasks described above. In particular, we consider a system with state $\mathbf{x} \in \mathbb{R}^n$, where n varies between 2 and 1024. The entries in the $n \times n$ matrix $\mathbf{A}$ are chosen randomly, according to a normal distribution (zero-mean with unit variance). The snapshots $\mathbf{x}_1, \dots, \mathbf{x}_m$ are also chosen to be random vectors, whose components are also chosen according to the standard normal distribution. In the tests below, we use a fixed number of snapshots $m = 10^4$. For minibatch DMD and windowed DMD, the window size is fixed at $w = 2048$, and online DMD and windowed DMD are both initialized after the first w snapshot pairs. For streaming DMD with a fixed rank r , we take $r = 16$. The simulations are performed in MATLAB (R2016b) on a personal computer equipped with a 2.6 GHz Intel Core i5 processor.

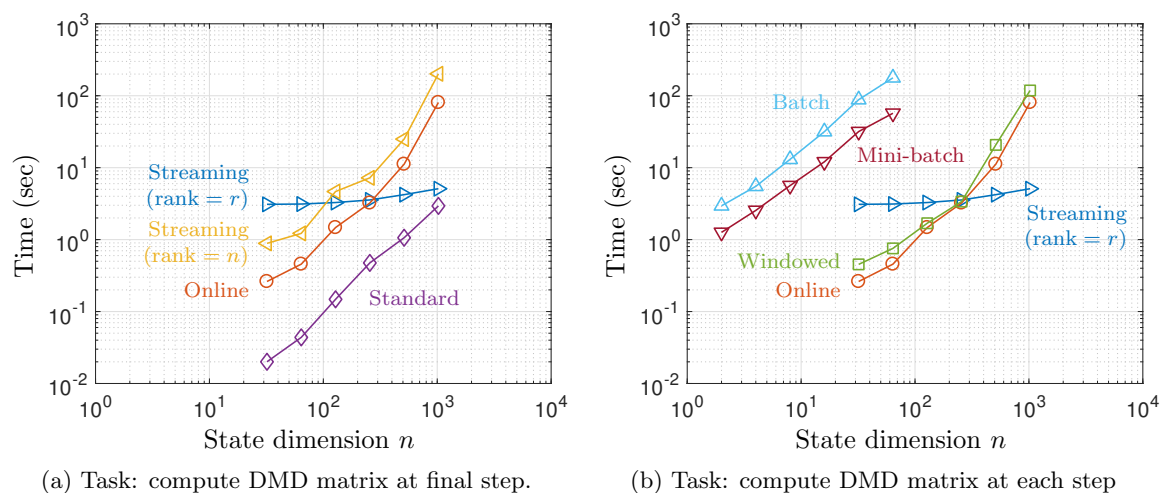


Figure 3. Performance of the different DMD algorithms on the benchmark cases described in subsection 5.1. For low-rank streaming, the dimension is limited to $r = 16$.

The results are shown in Figures 3(a) and 3(b). For the first task (computing the DMD matrix only at the final step), the standard DMD algorithm is the most efficient, for the problem sizes considered here. However, note that streaming DMD with a fixed rank r scales much better with the state dimension n and would be the fastest approach for problems with larger state dimension.

Our primary interest here is in the second task, shown in Figure 3(b), in which the DMD matrix is updated at each step. For problems with $n < 256$, online DMD is the fastest approach and can be orders of magnitude faster than the standard batch and minibatch algorithms. For problems with larger state dimension, streaming DMD is the fastest algorithm, as it scales linearly in the state dimension (while the other algorithms scale quadratically). However, note that streaming DMD does not compute the exact DMD matrix: rather, it computes a projection onto a subspace of dimension r (here 16). By contrast, online DMD and windowed DMD both compute the full DMD matrix, without approximation.

These results focus on the time required for these algorithms, but it is worth pointing out the memory requirements as well. Streaming DMD and online DMD do not require storage of any past snapshots, while windowed DMD and mini-batch DMD require storing the w snapshots in the window, and batch DMD requires storage of all past snapshots.

5.2. Linear time-varying system. We now test the online DMD and windowed DMD algorithms on a simple linear system that is slowly varying in time. In particular, consider the system

$$(34a) \quad \dot{\mathbf{x}}(t) = \mathbf{A}(t)\mathbf{x}(t),$$

where $\mathbf{x}(t) \in \mathbb{R}^2$, and the time-varying matrix $\mathbf{A}(t)$ is given by

$$(34b) \quad \mathbf{A}(t) = \begin{bmatrix} 0 & \omega(t) \\ -\omega(t) & 0 \end{bmatrix},$$

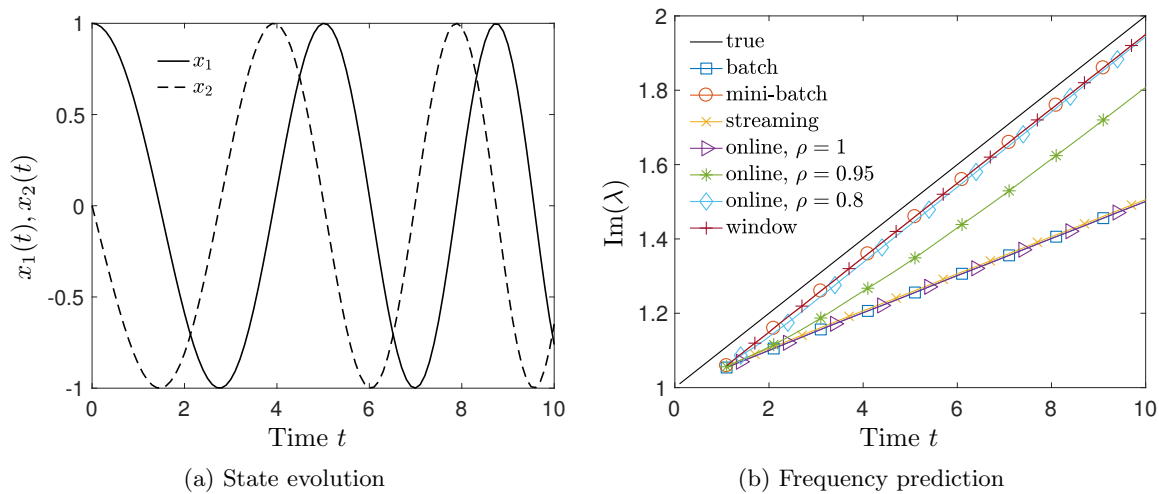


Figure 4. Solution of the linear time-varying system (34), and frequencies predicted by various DMD algorithms. For minibatch and windowed DMD, the window size is $w = 10$. For online DMD, smaller values of the parameter ρ result in faster tracking of the time-varying frequency.

where

$$\omega(t) = 1 + \epsilon t.$$

We take $\epsilon = 0.1$, so that the system is slowly varying in time. The eigenvalues of $\mathbf{A}(t)$ are $\pm i\omega(t)$, and it is straightforward to show that $\|\mathbf{x}(t)\|$ is constant in t . We simulate the system for $0 < t < 10$ from initial condition $\mathbf{x}(0) = (1, 0)^T$, and the snapshots are taken with time step $\Delta t = 0.1$ as shown in Figure 4(a). It is evident from the figure that the frequency is increasing with time.

Given the snapshots, we apply both brute-force batch DMD and minibatch DMD as benchmarks, then we compare streaming DMD, online DMD, and windowed DMD with these two benchmark brute-force algorithms. The finite time window size of minibatch DMD and windowed DMD is $w = 10$. Batch DMD takes into account all the past snapshots, while minibatch DMD only takes the recent snapshots from a finite time window. Streaming DMD, online DMD, and windowed DMD are initialized using the first $w = 10$ snapshot pairs, and they start iteration from time $w + 1$. Batch DMD and minibatch DMD also start from time $w + 1$. The results for streaming DMD, online DMD ($\rho = 1, \rho = 0.95, \rho = 0.8$), and windowed DMD are shown in Figure 4(b). DMD finds the discrete-time eigenvalues μ_{DMD} from data, and the figure shows the continuous-time DMD eigenvalues λ_{DMD} , which are related to these by

$$(35) \quad \mu_{\text{DMD}} = e^{\lambda_{\text{DMD}} \Delta t},$$

where Δt is the time spacing between snapshot pairs. We show the DMD results starting from time $w + 1$, and the true eigenvalues are also shown for comparison.

Observe from Figure 4(b) that the eigenvalues computed by the standard algorithm (batch DMD) agree with those identified by streaming DMD and online DMD with $\rho = 1$, as expected. Similarly, windowed DMD perfectly overlaps with minibatch DMD. When the weighting ρ in

online DMD is smaller than 1, the identified frequencies shift slightly toward those identified by windowed DMD. If we further decrease the weighting factor ($\rho = 0.8$), online DMD aggressively forgets old data, and the identified frequency adapts more quickly. This example demonstrates that windowed DMD and weighted online DMD are capable of capturing time variations in dynamics, with an appropriate choice of the weight ρ . A MATLAB script implementing this example can be found in [34]

5.3. Pressure fluctuations in a separation bubble. We now demonstrate the algorithm on a more complicated example, using data obtained from a wind tunnel experiment. In particular, we study the flow over a flat plate with an adverse pressure gradient and investigate the dynamics of pressure fluctuations in the vicinity of a separation bubble.

The setup of the wind tunnel experiment is shown in Figure 5(a). A flat plate with an elliptical (4:1) leading edge is placed in the flow, and zero-net mass flux suction and blowing are applied at the ceiling of the wind tunnel in order to impose an adverse and subsequent favorable pressure gradient along the surface of the plate, causing the boundary layer to separate and then reattach. More information about the separation system and the plate model can be found in [6].

These experiments were conducted in the Florida State Flow Control open-return wind tunnel. The cross-sectional dimensions of the test section are $30.5 \text{ cm} \times 30.5 \text{ cm}$, and the length is 61.0 cm . The contraction ratio of the inlet is 9:1. An aluminum honeycomb mesh and two fine, antiturbulence screens condition the flow at the inlet and provide a freestream turbulence intensity of 0.5%. The suction/blowing on the ceiling of the wind tunnel test section is provided by a variable-speed fan mounted within a duct fixed to the ceiling of the wind tunnel, which pulls flow from the ceiling and reintroduces it immediately downstream. The chord of the flat plate model is $c = 40.2 \text{ cm}$, and the height is $h = 0.095c$. For these experiments, the freestream velocity is $U_\infty = 3.9 \text{ m/s}$ and the Reynolds number is $Re_c = U_\infty c / \nu = 10^5$.

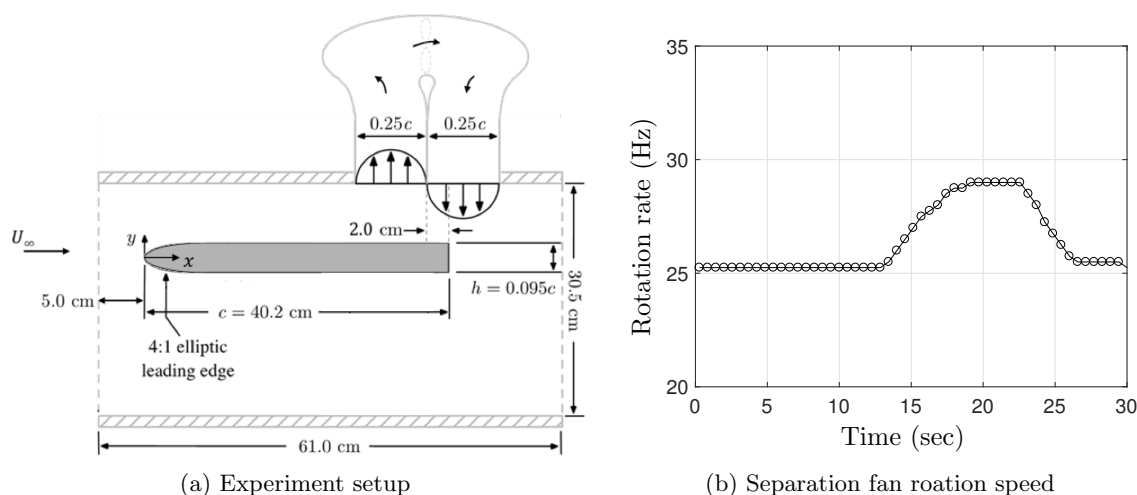


Figure 5. Schematic of the flat plate model and flow separation system. Separation fan rotation speed (rotation per second).

Unsteady surface pressure fluctuations within the separated flow are monitored by an array of 13 surface-mounted Panasonic WM-61A electret microphones located within the separation region. The microphones are placed at the centerline of the plate, between $x/c = 0.70$ and 0.94 , with a spacing of $\Delta x/c = 0.02$. More details regarding this microphone array can be found in [6]. These 13 microphone signals provide the data we use for online DMD.

We change the rotation speed of the separation fan slowly, in order to impose a time-varying boundary condition and induce time-varying dynamics in the separation bubble. The size of the separation bubble increases with the rotation speed of the separation fan. The rotation speed starts at about 25 Hz (rotation per second) and increases slowly to reach a high of about 30 Hz, then back to initial speed of 25 Hz. The blade rotation rate is shown in Figure 5(b). The pressure data sampling frequency is $f_s = 10^4$, and we collect data for $T = 30$ sec. The total number of samples is $m = 3 \times 10^5$, and the state dimension is $n = 13$ (there are 13 pressure sensors).

To study the frequencies from the pressure data, we first present a spectral analysis of the pressure data using the short-time discrete Fourier transform. Figure 6 shows the results for the seventh (medium) pressure sensor; other pressure sensors have similar results. A window size of $f_s = 10^4$ is used (along with a Hamming window of the same length), with 90% overlap of samples between adjoining sections. It's observed that there is no persistent peak frequency, and the dominant frequencies are time-varying. In order to gain a comprehensive understanding of the frequency variations in all the pressure sensors, and how these might be related to one another, we apply online DMD and windowed DMD to all the pressure data.

The number of snapshots $m = 3 \times 10^5$ is much larger than the state dimension $n = 13$, so the overconstrained assumption is satisfied. Recall that our proposed algorithm relies on the assumption that the number of snapshots is larger than the state dimension. Therefore, we can apply both online DMD and windowed DMD for this problem. The dynamics of the pressure fluctuations can be characterized by the DMD frequencies, which may be slowly varying in time. The DMD frequency is defined as

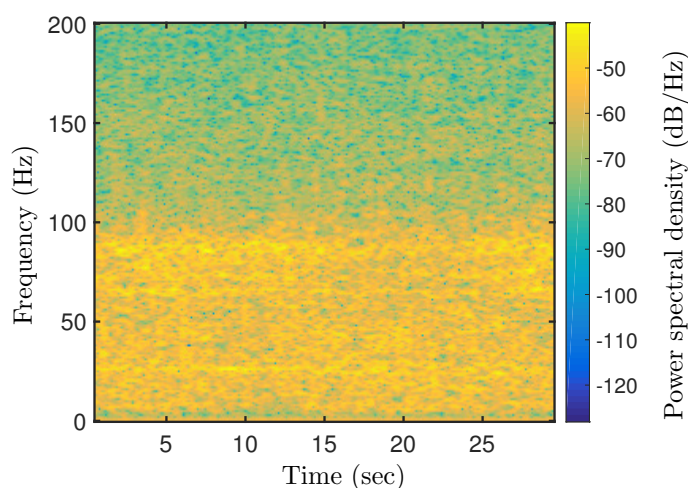


Figure 6. Power spectral density of the pressure measurement at the seventh (medium) pressure sensor. There is no persistent peak frequency, and the dominant frequencies are time-varying.

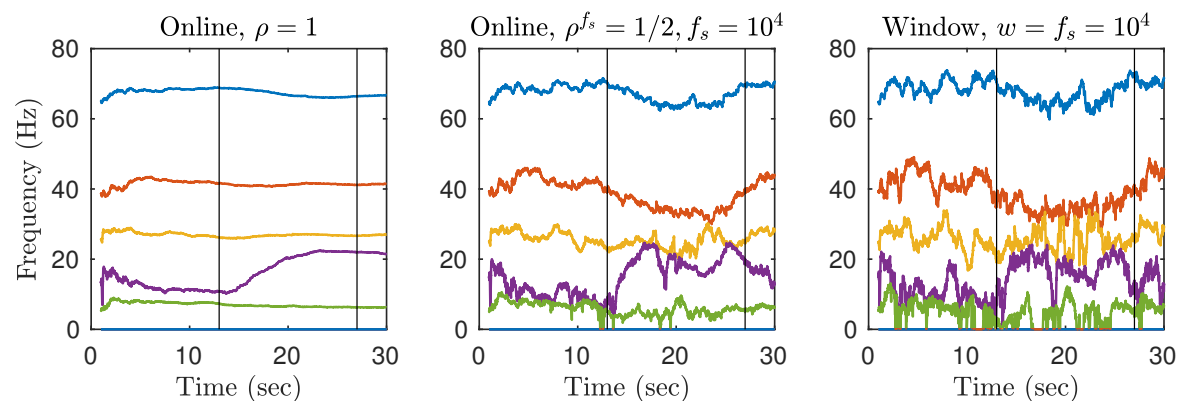


Figure 7. DMD frequencies identified by different DMD algorithms from 13 pressure signals, as described in subsection 5.3. The two vertical lines indicate the time when the fan rotation rate starts to increase ($t = 13$ sec) and finally settles down to the original speed ($t = 27$ sec).

$$f_{\text{DMD}} = \frac{\text{Im}(\lambda_{\text{DMD}})}{2\pi},$$

where $\text{Im}(\lambda_{\text{DMD}})$ is the imaginary part of the continuous-time DMD eigenvalues computed from (35). (The discrete-time eigenvalues μ_{DMD} are eigenvalues of the 13×13 matrix matrix $\mathbf{A}_k$.) The frequencies computed by the various DMD algorithms are shown in Figure 7. There are 13 DMD eigenvalues in total, and one of them is $f_0 = 0$ Hz corresponding to the mean. The remaining DMD eigenvalues consist of six complex-conjugate pairs, corresponding to six nonzero DMD frequencies. The DMD computation reveals three zero frequencies for all cases we run, indicating that there are only five active (nonzero) frequencies present in the system. Online DMD with $\rho = 1$ is run to obtain the standard DMD result. We use a weighting factor of $\rho^{f_s} = 0.5$ (corresponding half-decay life is f_s samples, i.e., one second.) for weighted online DMD and window size of $w = f_s = 10^4$ for windowed DMD. The DMD algorithms are initialized with the first w snapshots, and the resulting DMD frequencies are computed starting from time step $w + 1$.

Recall that with $\rho = 1$, online DMD coincides with the standard DMD algorithm. From Figure 7, we see that for this case, the three leading frequencies remain more or less constant in time. However, we expect the dynamics in the separation bubble to be time-varying. In response to the change of fan speed, the frequencies should start to change at $t = 13$ sec and return close to their original values at $t = 27$ sec (with some lag). The weighted online DMD and windowed DMD both act as desired, while the standard DMD is clearly erroneous. With weighting factor $\rho^{f_s} = 1/2$, online DMD behaves more like windowed DMD: in particular, the method is better at tracking variations in the frequency. The weighting factor in online DMD acts like a soft cutoff for the old snapshots, compared with the hard cutoff imposed by windowed DMD. Online DMD gradually forgets the old snapshots, so it is expected to be smoother than windowed DMD. From the results, we confirm that the online DMD is smoother compared to the windowed DMD. While the frequency variations shown in Figure 7 appear to be rapid or noisy, note that the time interval shown in the figure is quite long (about 300 periods for the lowest frequency of around 10 Hz), so it is reasonable to consider these frequencies as slowly varying in time.

Due to the slowly time-varying boundary condition imposed by the separation fan, the dynamics of the pressure fluctuations in the separation bubble is expected to be slowly time-varying too. When we simply use standard DMD, then the frequencies (and linear models) we identify will not be adaptive to the flow conditions in real time. In contrast, weighted online DMD and windowed DMD successfully track the time variations from the pressure signals. Therefore, the proposed online algorithms are demonstrated to be beneficial in real-time modeling of time-varying systems. In fact, the online DMD algorithm has been applied to this particular separation control problem [7] for system identification and real-time control purposes. The idea is to use online DMD to find an adaptive linear model from pressure measurements and control input history (see section 4.1) and apply linear control based on this adaptive model.

6. Conclusion and outlook. In this work, we have developed efficient methods for computing online DMD and windowed DMD. The proposed algorithms are especially useful in applications for which the number of snapshots is very large compared to the state dimension and when the dynamics are slowly varying in time. The number of snapshots is assumed to be larger than state dimension, because we are primarily interested in real-time modeling (updating the DMD matrix). In real time, it is often the case that we have a relatively small number of measurements, and as time progresses, eventually we will always have more snapshots than measurements. A weighting factor can be included easily in the online DMD algorithm, which is used to weight recent snapshots more heavily than older snapshots. This approach corresponds to using a soft cutoff for older snapshots, while windowed DMD uses a hard cutoff, from a finite time window. The proposed algorithms can be readily extended to online system identification, even for time-varying systems.

The efficiency is compared against the standard DMD algorithm, both for situations in which one computes the DMD matrix only at the final time and for situations in which one computes the DMD matrix in an “online” manner, updating it as new snapshots become available. The latter case is applicable, for instance, when one expects the dynamics to be time varying. For the former case, the standard DMD algorithm is the most efficient, while for the latter case, the new online and windowed DMD algorithms are the most efficient and can be orders of magnitude more efficient than the standard DMD algorithm. Table 1 provides a brief comparison of the main characteristics and features of standard DMD, batch DMD, minibatch DMD, (low rank) streaming DMD, online DMD, and windowed DMD.

The algorithms are further demonstrated on a number of examples, including a linear time-varying system, and data obtained from a wind tunnel experiment. As expected, weighted online DMD and windowed DMD are effective at capturing time-varying dynamics.

In this paper we assign different weights to different samples, and in particular a weighting factor is used to forget old snapshots. This choice of weight is especially useful for time-varying systems, as older information becomes outdated. As one reviewer pointed out, some real-world applications might involve measurements with different degrees of reliability (e.g., different levels of uncertainty, different measurement methods). One could in principle place different weights on different samples, according to how much one “trusts” them, which would lead to a weighted linear regression problem. This could be an interesting direction for future work.

Table 1

Characteristics of the various DMD algorithms considered. Relevant parameters are state dimension n , total number of snapshot pairs $m \gg n$, window size w such that $n < w \ll m$, low rank $r < n$, and discrete time $k > n$. Computational time denotes the required floating-point multiplies for one iteration (computing the DMD matrix).

Aspect	Standard	Batch	Minibatch	Streaming	Online	Windowed
Computational time	$\mathcal{O}(mn^2)$	$\mathcal{O}(kn^2)$	$\mathcal{O}(wn^2)$	$\mathcal{O}(r^2n)$	$4n^2$	$8n^2$
Memory	mn	kn	wn	$\mathcal{O}(rn)$	$2n^2$	$wn + 2n^2$
Store past snapshots	Yes	Yes	Yes	No	No	Yes
Track time variations	No	No	Yes	Yes	Yes	Yes
Real-time DMD matrix	No	Yes	Yes	Yes	Yes	Yes
Exact DMD matrix	Yes	Yes	Yes	No	Yes	Yes

Another relevant direction for future work is a more detailed study of the application of proposed online/windowed DMD algorithms to system identification. In situations where there are variations in dynamics, or where we desire real-time control, it is crucial to build accurate and adaptive reduced order models for effective control, and the methods proposed here could be useful in these cases.

Acknowledgment. The authors thank Professor Maziar S. Hemati for helpful suggestions about references.

REFERENCES

- [1] E. L. ALLOWER AND K. GEORG, *Update methods and their numerical stability*, in Numerical Continuation Methods, Springer Ser. Comput. Math. 13, Springer, New York, 1990, pp. 252–265, https://doi.org/10.1007/978-3-642-61257-2_16.
- [2] K. J. ÅSTRÖM AND P. EYKHOFF, *System identification—a survey*, Automatica, 7 (1971), pp. 123–162, [https://doi.org/10.1016/0005-1098\(71\)90059-8](https://doi.org/10.1016/0005-1098(71)90059-8).
- [3] R. BELLMAN AND J. M. RICHARDSON, *On some questions arising in the approximate solution of nonlinear differential equations*, Quart. Appl. Math., 20 (1963), pp. 333–339, <https://doi.org/10.1090/qam/144472>.
- [4] C. M. BISHOP, *Pattern Recognition and Machine Learning*, Springer, New York, 2006.
- [5] S. L. BRUNTON, J. L. PROCTOR, AND J. N. KUTZ, *Discovering governing equations from data by sparse identification of nonlinear dynamical systems*, Proc. Natl. Acad. Sci. USA, 113 (2016), pp. 3932–3937, <https://doi.org/10.1073/pnas.1517384113>.
- [6] E. DEEM, L. CATTAFESTA, H. ZHANG, C. ROWLEY, M. HEMATI, F. CADIEUX, AND R. MITTAL, *Identifying dynamic modes of separated flow subject to ZNMF-based control from surface pressure measurements*, in Proceedings of the 47th AIAA Fluid Dynamics Conference, 2017, <https://doi.org/10.2514/6.2017-3309>.
- [7] E. A. DEEM, L. N. CATTAFESTA, H. YAO, M. HEMATI, H. ZHANG, AND C. W. ROWLEY, *Experimental implementation of modal approaches for autonomous reattachment of separated flows*, in Proceedings of the AIAA Aerospace Sciences Meeting, 2018, p. 1052.
- [8] Z. DRMAC, I. MEZIC, AND R. MOHR, *Data driven modal decompositions: analysis and enhancements*, SIAM J. Sci. Comput., 40 (2018), pp. A2253–A2285, <https://doi.org/10.1137/17M1144155>.
- [9] N. B. ERICHSON, S. L. BRUNTON, AND J. N. KUTZ, *Randomized Dynamic Mode Decomposition*, preprint, [arXiv:1702.02912](https://arxiv.org/abs/1702.02912), 2017.

- [10] N. B. ERICHSON AND C. DONOVAN, *Randomized low-rank dynamic mode decomposition for motion detection*, Comput. Vision Image Understanding, 146 (2016), pp. 40–50, <https://doi.org/10.1016/j.cviu.2016.02.005>.
- [11] A. GERMANI, C. MANES, AND P. PALUMBO, *Polynomial extended kalman filter*, IEEE Trans. Automat. Control, 50 (2005), pp. 2059–2064, <https://doi.org/10.1109/TAC.2005.860256>.
- [12] J. GROSEK AND J. N. KUTZ, *Dynamic Mode Decomposition for Real-Time Background/Foreground Separation in Video*, preprint, [arXiv:1404.7592](https://arxiv.org/abs/1404.7592), (2014).
- [13] W. W. HAGER, *Updating the inverse of a matrix*, SIAM Rev., 31 (1989), pp. 221–239, <https://doi.org/10.1137/1031049>.
- [14] M. S. HEMATI, S. T. DAWSON, AND C. W. ROWLEY, *Parameter-varying aerodynamics models for aggressive pitching-response prediction*, AIAA J., 55 (2017), pp. 1–9, <https://doi.org/10.2514/1.J055193>.
- [15] M. S. HEMATI, E. A. DEEM, M. O. WILLIAMS, C. W. ROWLEY, AND L. N. CATTAFESTA, *Improving Separation Control with Noise-Robust Variants of Dynamic Mode Decomposition*, in Proceedings of the 54th AIAA Aerospace Sciences Meeting, 2016, <https://doi.org/10.2514/6.2016-1103>.
- [16] M. S. HEMATI, M. O. WILLIAMS, AND C. W. ROWLEY, *Dynamic mode decomposition for large and streaming datasets*, Phy. Fluids, 26 (2014), 111701, <https://doi.org/10.1063/1.4901016>.
- [17] T. C. HSIA, *System Identification*, Lexington Books, Lanham, MD, 1977.
- [18] J. N. KUTZ, S. L. BRUNTON, B. W. BRUNTON, AND J. L. PROCTOR, *Dynamic Mode Decomposition: Data-Driven Modeling of Complex Systems*, SIAM, Philadelphia, 2016, <https://doi.org/10.1137/1.9781611974508>.
- [19] S. MAČEŠIĆ, N. ČRNJARIĆ-ŽIC, AND I. MEZIĆ, *Koopman Operator Family Spectrum for Nonautonomous Systems—Part 1*, preprint, [arXiv:1703.07324](https://arxiv.org/abs/1703.07324), 2017.
- [20] D. MATSUMOTO AND T. INDINGER, *On-the-Fly Algorithm for Dynamic Mode Decomposition Using Incremental Singular Value Decomposition and Total Least Squares*, preprint, [arXiv:1703.11004](https://arxiv.org/abs/1703.11004), 2017.
- [21] O. NELLES, *Nonlinear System Identification: From Classical Approaches to Neural Networks and Fuzzy Models*, Springer, New York, 2013, <https://doi.org/10.1007/978-3-662-04323-3>.
- [22] J. L. PROCTOR, S. L. BRUNTON, AND J. N. KUTZ, *Dynamic mode decomposition with control*, SIAM J. on Appl. Dyn. Syst., 15 (2016), pp. 142–161, <https://doi.org/10.1137/15M1013857>.
- [23] C. W. ROWLEY AND S. T. DAWSON, *Model reduction for flow analysis and control*, Annu. Rev. Fluid Mech., 49 (2017), pp. 387–417, <https://doi.org/10.1146/annurev-fluid-010816-060042>.
- [24] C. W. ROWLEY, I. MEZIĆ, S. BAGHERI, P. SCHLATTER, AND D. S. HENNINGSON, *Spectral analysis of nonlinear flows*, J. Fluid Mech., 641 (2009), pp. 115–127, <https://doi.org/10.1017/S0022112009992059>.
- [25] P. J. SCHMID, *Dynamic mode decomposition of numerical and experimental data*, J. Fluid Mech., 656 (2010), pp. 5–28, <https://doi.org/10.1017/S0022112010001217>.
- [26] J. SHERMAN AND W. J. MORRISON, *Adjustment of an inverse matrix corresponding to a change in one element of a given matrix*, Ann. Math. Statist., 21 (1950), pp. 124–127, <https://doi.org/10.1214/aoms/1177729893>.
- [27] J. H. TU, C. W. ROWLEY, D. M. LUCHTENBURG, S. L. BRUNTON, AND J. N. KUTZ, *On dynamic mode decomposition: Theory and applications*, J. Comput. Dyn., 1 (2014), pp. 391–421, <https://doi.org/10.3934/jcd.2014.1.391>.
- [28] V. VERDULT AND M. VERHAEGEN, *Kernel methods for subspace identification of multivariable LPV and bilinear systems*, Automatica, 41 (2005), pp. 1557–1565, <https://doi.org/10.1016/j.automatica.2005.03.027>.
- [29] M. O. WILLIAMS, I. G. KEVREKIDIS, AND C. W. ROWLEY, *A data-driven approximation of the Koopman operator: Extending dynamic mode decomposition*, J. Nonlinear Sci., 25 (2015), pp. 1307–1346, <https://doi.org/10.1007/s00332-015-9258-5>.
- [30] M. O. WILLIAMS, C. W. ROWLEY, AND I. G. KEVREKIDIS, *A kernel-based method for data-driven koopman spectral analysis*, J. Comput. Dyn., 2 (2015), pp. 247–265.
- [31] M. A. WOODBURY, *Inverting Modified Matrices*, Memorandum report, 42, Statistical Research Group, Princeton University, 1950.
- [32] E. L. YIP, *A note on the stability of solving a rank-p modification of a linear system by the Sherman–Morrison–Woodbury formula*, SIAM J. Sci. Stat. Comput., 7 (1986), pp. 507–513, <https://doi.org/10.1137/0907034>.

- [33] H. ZHANG, S. T. DAWSON, C. W. ROWLEY, E. A. DEEM, AND L. N. CATTAFESTA, *Evaluating the Accuracy of the Dynamic Mode Decomposition*, preprint, [arXiv:1710.00745](https://arxiv.org/abs/1710.00745), 2017.
- [34] H. ZHANG AND C. W. ROWLEY, *Online DMD and Window DMD Implementation in Matlab and Python*, <https://github.com/haozhg/odmd> (2017).